

Documentación Técnica para Auditoría Sistema de Gestión de Tickets IT

Auditoría Técnica

December 16, 2025

1 Portada Formal

SISTEMA DE GESTIÓN DE TICKETS PARA SOPORTE TÉCNICO IT

Documento: Documentación Técnica para Auditoría

Tecnologías Utilizadas:

- Backend: PHP 7.0+, PDO, MySQL
- Frontend: HTML5, CSS3, JavaScript Vanilla
- Framework CSS: Bootstrap 5.3.0
- Iconografía: FontAwesome 6.0.0
- Autenticación: Sesiones PHP, Hash de contraseñas
- Integración: Firebase Cloud Messaging (FCM), JWT
- Librerías PHP: PhpOffice/PhpSpreadsheet, Firebase/PHP-JWT, Google/Auth
- Base de Datos: MySQL con PDO

Fecha de Generación: December 16, 2025

2 Alcance del Documento

Esta documentación técnica para auditoría cubre la totalidad de componentes del sistema de gestión de tickets IT desarrollado en PHP, HTML, CSS y JavaScript. El alcance incluye: (1) módulos de autenticación y autorización, (2) interfaces de usuario para clientes públicos, usuarios autenticados y administradores IT, (3) endpoints de API para operaciones CRUD de tickets, (4) funcionalidades de reportería y exportación de datos, (5) integración con servicios de notificación por FCM, (6) gestión de archivos adjuntos, y (7) captura de satisfacción del usuario. No incluye configuración de infraestructura de servidor, variables de entorno sensibles o documentación de bases de datos separadas.

3 Descripción General del Sistema

El sistema implementa una solución centralizada de gestión de tickets de soporte técnico con arquitectura cliente-servidor. Opera bajo modelo de tres capas de usuarios: (1) clientes públicos sin autenticación que crean y consultan tickets mediante número de ficha, (2) usuarios autenticados del departamento IT que visualizan y asignan tickets, y (3) administradores IT con acceso completo a estadísticas, reportería y gestión de personal. La plataforma facilita la creación de solicitudes de soporte, asignación a personal técnico, seguimiento en tiempo real, adjunción de evidencia digital, y cierre con evaluación de satisfacción. Utiliza sesiones PHP para mantener contexto de usuario, base de datos MySQL para persistencia, y endpoints JSON para comunicación asincrónica.

4 Estructura del Proyecto

4.1 Organización de Directorios

El proyecto presenta la siguiente estructura de carpetas y archivos:

- / (raíz): Archivos principales de entrada y vistas renderizadas en servidor
 - `index.php`: Portal público de acceso sin autenticación
 - `login.php`: Interfaz de autenticación para usuarios IT
 - `dashboard.php`: Panel principal de usuarios autenticados
 - `admin.php`: Panel de administración para personal IT
 - `logout.php`: Manejador de cierre de sesión
 - `crear_usuarios.php`: Script de inicialización de usuarios administrativos
 - `api.php`: Enrutador principal de API con lógica centralizada
- /`config`: Configuración centralizada
 - `database.php`: Clase Database, funciones globales de sesión y utilidades
- /`api`: Endpoints especializados de API
 - `tickets.php`: CRUD de tickets para usuarios autenticados
 - `tickets_admin.php`: Operaciones administrativas de tickets
 - `tickets_publico.php`: Consultas y creación de tickets públicos
 - `reportes.php`: Generación de reportes en formato Excel
- /`vendor`: Dependencias de Composer
- `service-account-key.json`: Credenciales de Firebase Cloud Messaging
- `sistema_tickets_it.sql`: Esquema de base de datos inicial

5 Componentes Backend (PHP)

5.1 config/database.php

Módulo de configuración centralizada. Define clase Database que gestiona conexión PDO a MySQL con manejo de excepciones. Implementa funciones globales para: inicialización de sesiones, verificación de autenticación requerida, validación de rol IT, y formateo de fechas y colores según estado y prioridad de tickets.

```
class Database {
    public function getConnection() {
        $this->conn = new PDO(
            "mysql:host=" . $this->host . ";dbname=" . $this->
                db_name ,
            $this->username, $this->password
        );
        $this->conn->exec("set names utf8");
        return $this->conn;
    }
}
```

5.2 login.php

Interfaz de autenticación y lógica de validación de credenciales. Procesa solicitudes POST con email y contraseña, valida contra tabla usuarios, genera sesión con atributos usuario_id, nombre, email y rol. Actualiza timestamp de último acceso y retorna respuesta JSON con estado de éxito o mensaje de error.

```
if (password_verify($password, $row['password'])) {
    $_SESSION['usuario_id'] = $row['id'];
    $_SESSION['rol'] = $row['rol'];
    echo json_encode(['success' => true, 'rol' => $row['rol']]);
}
```

5.3 logout.php

Manejador de destrucción de sesión. Limpia variable global \$_SESSION, elimina cookie de sesión del navegador y redirige a login.php. Implementa protocolo estándar de cierre seguro de sesión.

```
\$_SESSION = array();
```

```
session_destroy();  
header("Location: login.php");
```

5.4 index.php

Portal público HTML renderizado en servidor para creación y consulta de tickets sin autenticación. Utiliza Bootstrap 5 y FontAwesome para interfaz visual. Implementa dos secciones con pestañas: identificación por número de ficha para consultar tickets existentes, y formulario para crear nuevo ticket con campos titulo, descripción, prioridad, categoría y archivos adjuntos. Incluye JavaScript incrustado para solicitudes asincrónicas AJAX a endpoints de API.

```
<input type="text" class="form-control" id="numeroFicha"  
      placeholder="Ingresa tu número de ficha">  
<button type="button" class="btn btn-primary" onclick="  
      buscarTickets()">  
      Buscar mis Tickets  
</button>
```

5.5 login.php (Vista HTML)

Interfaz de autenticación con formulario de email y contraseña. Contiene estilos CSS incrustados para gradiente de fondo, tarjeta de login con efecto blur y bordes redondeados. JavaScript manejador de submit realiza fetch a login.php con método POST, procesa respuesta JSON y redirige a dashboard según rol.

```
fetch('login.php', {  
  method: 'POST',  
  body: formData  
})  
.then(response => response.json())  
.then(data => {  
  if (data.success) {  
    window.location.href = 'dashboard.php';  
  }  
});
```

5.6 dashboard.php

Panel principal de usuarios autenticados (rol solicitante e IT). Renderiza navbar con menú dinámico según rol de sesión. Incluye secciones: (1) Mis Tickets con lista paginable, (2) Crear Ticket, (3) Detalles de Ticket en modal, (4) Personal IT disponible con indicadores de estado. Estilos CSS incrustados definen tarjetas animadas, colores por prioridad, línea de tiempo de eventos, y layout responsivo. JavaScript manejador de eventos para operaciones AJAX.

```
<nav class="navbar navbar-expand-lg navbar-dark navbar-custom">
  <div class="container-fluid">
    <a class="navbar-brand" href="#dashboard">
      <i class="fas fa-headset me-2"></i>
      <strong>Sistema IT</strong>
    </a>
  </div>
</nav>
```

5.7 admin.php

Panel de administración exclusivo para personal IT. Implementa autenticación separada (admin_id en sesión). Renderiza interfaz con secciones: (1) Tickets recientes, (2) Estadísticas globales, (3) Gestión de personal IT, (4) Reportería. Contiene estilos CSS para tarjetas de estadísticas con gradientes, tablas de datos, línea de tiempo de eventos de tickets. JavaScript para comunicación asincrónica con api/tickets_admin.php.

```
if (!isset($_SESSION['admin_id'])) {
    echo json_encode(['success' => false, 'message' => 'No
        autorizado']);
    exit();
}
```

5.8 crear_usuarios.php

Script de inicialización ejecutable desde CLI. Crea usuarios administrativos pre-configurados (Oscar Romo, Marcos Palomo, Gilberto Treviño) con rol 'it', email corporativo y contraseña hasheada. Utiliza función password_hash con algoritmo bcrypt. Diseñado para ejecución única en instalación del sistema.

```
$passwordHash = password_hash($usuario['password'],
    PASSWORD_DEFAULT);
```



```
$stmt->bindParam(1, $usuario['nombre']);
$stmt->bindParam(2, $usuario['email']);
```

5.9 api.php

Enrutador principal centralizado de API. Implementa: (1) Funciones de conexión a base de datos PDO, (2) Logging en tiempo real a archivo api_debug.log, (3) Gestión de tokens FCM (Firebase Cloud Messaging), (4) Autenticación JWT simplificada, (5) Endpoints para operaciones de tickets, usuarios, estadísticas, y notificaciones push, (6) Gestión de cargas de archivos adjuntos. Retorna todas las respuestas en formato JSON con headers CORS configurados.

```
function get_db_connection() {
    $dsn = "mysql:host={$db_config['host']};dbname={$db_config['database']}";
    $pdo = new PDO($dsn, $db_config['username'], $db_config['password']);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    return $pdo;
}
```

5.10 api/tickets.php

Endpoint especializado para operaciones de tickets bajo autenticación requerida (verificarSesion). Implementa método de router según REQUEST_METHOD (GET, POST) y parámetro action. Funciones: obtenerMisTickets (lista propia del usuario), obtenerTodosTickets (acceso IT), obtenerEstadisticas (contadores por estado), obtenerPersonalIT (lista de personal disponible), obtenerDetalleTicket, crearTicket, tomarTicket (asignación), resolverTicket, cerrarTicket. Retorna JSON.

```
case 'GET':
    if ($action === 'mis-tickets') {
        obtenerMisTickets($db);
    } elseif ($action === 'estadisticas') {
        obtenerEstadisticas($db);
    }
    break;
```

5.11 api/tickets_admin.php

Endpoint para operaciones administrativas exclusivas del personal IT. Valida presencia de `$_SESSION['admin_id']`. Funciones: `obtenerTicketsRecientes` (limite 50), `obtenerTodosTickets`, `obtenerEstadisticas` (métricas globales), `obtenerDetalleTicket`, `tomarTicket` (asignación), `resolverTicket` (cambio de estado a resuelto). Respuestas en JSON con estructura `success/data`.

```
function obtenerTodosTickets(\$db) {
    \ $query = "SELECT t.*, u_asig.nombre as asignado_nombre
                FROM tickets t
                LEFT JOIN usuarios u_asig ON t.asignado_a = u_asig.id
                ORDER BY t.fecha_creacion DESC";
}
```

5.12 api/tickets_publico.php

Endpoint sin autenticación para operaciones públicas. Funciones: `obtenerTicketsPorFicha` (búsqueda por número_ficha y nombre), `obtenerDetalleTicket`, `verificarNombresSimilares` (búsqueda aproximada), `obtenerPersonalIT` (lista pública), `crearTicket` (formulario público). Valida parámetros obligatorios y retorna JSON. Incluye manejo de archivos adjuntos.

```
if (empty(\ $numeroFicha) || empty(\ $nombre)) {
    echo json_encode(['success' => false,
        'message' => 'Faltan datos de identificaci n']);
    return;
}
```

5.13 api/reportes.php

Generador de reportes en formato Excel mediante PhpOffice/PhpSpreadsheet. Requiere autenticación admin (`session['admin_id']`). Parámetros: `tipo` (general/estado/fecha), `filtro_estado`, `fecha_inicio`, `fecha_fin`. Construye consulta SQL dinámicamente, exporta datos a archivo .xlsx con encabezados, estilos de color y ajuste automático de columnas. Descarga el archivo al cliente.

```
$spreadsheet = new Spreadsheet();
$sheet = $spreadsheet->getActiveSheet();
$headers = ['A1' => 'ID', 'B1' => 'N mero Ficha', 'C1' => '
    Solicitante'];
```

```
$writer = new Xlsx(\$spreadsheet);
```

6 Componentes Frontend (Vistas HTML)

6.1 index.php (Interfaz Pública)

Página de entrada al sistema sin requerimiento de autenticación. Renderiza encabezado con título, alerta informativa sobre identificación correcta de usuario. Contiene dos secciones tabuladas: (1) Identificación: campos de número de ficha y nombre para consultar tickets existentes, (2) Crear Nuevo Ticket: formulario con título, descripción, prioridad (urgente/alta/media/baja), categoría, archivos adjuntos múltiples. Toda la interacción es asincrónica mediante JavaScript AJAX.

```
<div class="col-12 col-lg-10">
  <div class="card main-card">
    <div class="card-body p-4">
      <label class="form-label fw-bold">
        Número de Ficha *
      </label>
      <input type="text" class="form-control"
        id="numeroFicha" placeholder="Ingresa ficha">
    </div>
  </div>
</div>
```

6.2 login.php (Interfaz de Autenticación)

Página de login centrada verticalmente. Contiene tarjeta blanca traslúcida con icono de auriculares, título "Sistema IT", campo email con ícono de sobre, campo password con ícono de candado, botón de envío con ícono de entrada. Pie con usuarios de prueba pre-configurados. Validación del lado del cliente y comunicación AJAX a login.php.

```
<div class="col-md-6 col-lg-4">
  <div class="card login-card">
    <div class="input-group">
      <span class="input-group-text">
        <i class="fas fa-envelope"></i>
      </span>
      <input type="email" class="form-control"
        id="email" name="email" placeholder="Email">
    </div>
  </div>
</div>
```

6.3 dashboard.php (Panel de Usuarios Autenticados)

Interfaz renderizada en servidor para usuarios con sesión activa. Navbar responsivo con brand y menú de navegación dinámico (IT vs Solicitante). Secciones principales: (1) Resumen de estadísticas con contadores en tarjetas coloridas, (2) Lista de Mis Tickets con paginación, (3) Crear Ticket (modal), (4) Detalle de Ticket (modal con línea de tiempo), (5) Personal IT Disponible (lista con indicadores de estado). CSS para responsive design, animaciones de tarjetas, colores codificados por prioridad/estado.

```
<div class="stat-card">
  <div class="card-body">
    <h6 class="card-title">Tickets Abiertos</h6>
    <h2 id="abiertos">0</h2>
  </div>
</div>
```

6.4 admin.php (Panel Administrativo)

Interfaz exclusiva para personal IT con autenticación separada (admin_id). Renderiza: (1) Login de administrador si no está autenticado, (2) Panel completo con navbar y menú lateral, (3) Estadísticas globales, (4) Tabla de tickets recientes con opciones de asignación, (5) Gestión de personal IT, (6) Acceso a reportería. Incluye modal para crear/editar usuarios, modal de detalles de ticket con línea de tiempo de eventos, botones de exportación de reportes.

```
<form id="loginForm" method="POST">
  <input type="email" class="form-control"
    name="email" placeholder="Email" required>
  <input type="password" class="form-control"
    name="password" placeholder="Contrase a" required>
  <button type="submit" class="btn btn-primary w-100">
    Iniciar Sesi n
  </button>
</form>
```

7 Componentes JavaScript

7.1 index.php (JavaScript Público)

Funciones incrustadas para operaciones de tickets públicos: `buscarTickets()` (consulta por `número_ficha` y `nombre`), `crearTicket()` (envío de formulario con archivos), `obtenerPersonalIT()` (lista de disponibles). Manejo de eventos `submit`, validación de campos requeridos, construcción de `FormData` para multipart, procesamiento de respuesta JSON, actualización del DOM con resultados, gestión de errores con alertas visuales.

```
document.getElementById('loginForm').addEventListener('submit',
  function(e) {
    e.preventDefault();
    const formData = new FormData(this);
    fetch('login.php', {
      method: 'POST',
      body: formData
    })
    .then(response => response.json());
  }
);
```

7.2 login.php (JavaScript de Autenticación)

Manejador de evento `submit` en formulario de login. Captura datos de email y password, realiza `fetch POST` a `login.php`, procesa respuesta JSON, muestra mensajes de estado (cargando, éxito, error), redirige a `dashboard.php` si autenticación es exitosa, muestra error descriptivo en caso contrario.

```
document.getElementById('loginForm').addEventListener('submit',
  function(e) {
    e.preventDefault();
    fetch('login.php', {method: 'POST', body: formData})
      .then(r => r.json())
      .then(d => {
        if (d.success)
          window.location.href = 'dashboard.php';
      });
  }
);
```

7.3 dashboard.php (JavaScript de Panel de Usuario)

Conjunto extenso de funciones para gestión de tickets: `cargarTickets()` (GET a `api/tickets.php?action=mis-tickets`), `cargarEstadisticas()` (GET estadísticas), `crearTicket()` (POST crear), `tomarTicket()` (asignación), `resolverTicket()` (cambio de estado), `cerrarTicket()` (cierre con satisfacción), `cargarPersonalIT()` (disponibilidad). Manejo de modales Bootstrap, actualización dinámica de DOM, validación de campos, construcción y envío de `FormData` con archivos adjuntos, procesamiento de respuesta JSON.

```
function cargarTickets() {
    fetch('api/tickets.php?action=mis-tickets')
        .then(response => response.json())
        .then(data => {
            // Renderizar tickets en tabla
        });
}
```

7.4 admin.php (JavaScript Administrativo)

Funciones para panel administrativo: `cargarTickets()` (GET `api/tickets_admin.php?action=todos`), `cargarEstadisticas()` (métricas globales), `gestionarUsuarios()` (CRUD de usuarios IT), `asignarTicket()` (POST tomar), `cambiarEstado()` (POST resolver), `generarReporte()` (descarga Excel). Gestión de modales para detalles de ticket con línea de tiempo de eventos, validación de entrada, envío POST con parámetros, redirección para descarga de archivos Excel.

```
function cargarTickets() {
    fetch('api/tickets_admin.php?action=todos')
        .then(r => r.json())
        .then(d => {
            // Renderizar tabla de tickets administrativo
        });
}
```

8 Componentes de Estilo (CSS)

8.1 Estilos Globales (Incrustados en index.php)

Defines variables CSS de colores corporativos (primary: #667eea, secondary: #764ba2, success: #28a745, warning: #ffc107, danger: #dc3545). Body con gradiente de fondo lineal 135deg entre primary y secondary, altura mínima 100vh. Main-card con background translúcido (rgba), filtro blur de 10px, border-radius 20px, shadow 20px 40px. Ticket-card con transición de 0.2s y transform translateY(-2px) en hover. Reglas específicas para tarjetas de prioridad (border-left 5px sólido con color según nivel).

```
:root {
  --primary-color: #667eea;
  --secondary-color: #764ba2;
  --success-color: #28a745;
}
body {
  background: linear-gradient(135deg, var(--primary-color),
    var(--secondary-color));
  min-height: 100vh;
}
```

8.2 Estilos de Navbar (login.php y dashboard.php)

Navbar personalizado con background gradiente lineal 135deg (primary a secondary), shadow 0 2px 10px rgba(0,0,0,0.1). Toggle responsivo para dispositivos móviles. Colores de texto white en navbar-dark. Elemento brand con icono FontAwesome y nombre sistema.

```
.navbar-custom {
  background: linear-gradient(135deg, var(--primary-color),
    var(--secondary-color));
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
```

8.3 Estilos de Tarjetas de Dashboard (dashboard.php y admin.php)

Card-dashboard con border none, border-radius 15px, shadow 0 5px 15px rgba(0,0,0,0.08), transición transform 0.3s ease. En hover: translateY(-5px). Stat-card con background gradiente (primary a secondary), color white, border-radius 15px. Layout responsivo con media queries para dispositivos móviles.


```
.card-dashboard {
    border: none;
    border-radius: 15px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.08);
    transition: transform 0.3s ease;
}
.card-dashboard:hover {
    transform: translateY(-5px);
}
```

8.4 Estilos de Estado y Prioridad (dashboard.php)

Clases para coloración de tickets según prioridad: ticket-priority-urgente (border-left danger), ticket-priority-alta (border-left warning), ticket-priority-media (border-left primary), ticket-priority-baja (border-left secondary). Status-badge para etiquetas de estado con font-size 0.75rem, padding 0.35rem 0.65rem. User-status con círculo inline de 10px, border-radius 50

```
.ticket-priority-urgente {
    border-left: 5px solid var(--danger-color);
}
.user-status {
    display: inline-block;
    width: 10px;
    height: 10px;
    border-radius: 50%;
}
```

8.5 Estilos de Timeline (dashboard.php y admin.php)

Timeline con position relative, padding-left 30px. Timeline-item con position relative, margin-bottom 20px. Timeline-marker: absolute, left -35px, top 5px, width 12px, height 12px, border-radius 50

```
.timeline {
    position: relative;
    padding-left: 30px;
}
.timeline::before {
    content: '';
```

```
position: absolute;
left: -30px;
width: 2px;
background: #dee2e6;
}
```

8.6 Estilos de Formularios (Globales)

Form-control y form-select con border-radius 10px, padding 12px 15px. Botones con border-radius 10px, padding 10px 20px. Input-group con iconografía FontAwesome. Responsividad para pantallas pequeñas: media query max-width 768px ajusta propiedades de elementos flotantes.

```
.form-control, .form-select {
    border-radius: 10px;
    padding: 12px 15px;
}
.btn {
    border-radius: 10px;
    padding: 10px 20px;
}
```

9 Integración y Flujos de Datos

9.1 Flujo de Autenticación

Usuario accede a login.php, completa formulario email/password, JavaScript submit realiza POST a login.php. Backend valida credenciales contra tabla usuarios (hash bcrypt). Si válida: crea sesión con usuario_id, nombre, email, rol. Respuesta JSON con success=true, frontend redirige a dashboard.php. Si inválida: retorna mensaje de error. Sesión persiste en cookie del navegador.

9.2 Flujo de Creación de Ticket Público

Usuario en index.php completa formulario (ficha, nombre, título, descripción, prioridad, categoría, archivos). JavaScript construye FormData multipart con campos y File objects. POST a api/tickets_publico.php?action=crear. Backend valida parámetros, inserta en tabla tickets, procesa archivos adjuntos en directorio uploads/tickets/, retorna JSON con ticket_id. Frontend muestra confirmación.

9.3 Flujo de Asignación de Ticket

Usuario IT en dashboard.php visualiza lista de tickets abiertos. Click en botón "Tomar" realiza POST a api/tickets.php?action=tomar&id=X con asignado_a = usuario_id. Backend actualiza campo asignado_a en tabla tickets, cambia estado a en_proceso, retorna JSON. Frontend actualiza tabla y estadísticas. Notificación FCM opcional a cliente.

9.4 Flujo de Reportería

Admin en admin.php selecciona parámetros (tipo, estado, fechas), click en "Descargar Reporte". Navegador abre GET api/reportes.php?tipo=general&estado=abiertos. Backend construye consulta SQL con filtros, genera Spreadsheet con PhpOffice, serializa a .xlsx, retorna con headers Content-Type y Content-Disposition. Navegador descarga archivo.

10 Consideraciones de Seguridad y Arquitectura

10.1 Autenticación y Autorización

Sistema implementa autenticación basada en sesión PHP. Contraseñas almacenadas con hash bcrypt (PASSWORD_DEFAULT). Roles definidos: solicitante, it, admin. Funciones verificarSesion() y esIT() validan acceso a recursos protegidos. Panel administrativo requiere \$_SESSION['admin_id'] separado. API endpoints validan presencia de sesión activa. No se utiliza JWT para acceso de sesión (solo para FCM).

10.2 Control de Acceso

Usuarios solicitante ven solo sus propios tickets. Personal IT visualiza todos. Administradores acceden a reportería y gestión de usuarios. Endpoints protegidos evalúan \$_SESSION['rol'] antes de retornar datos. Parámetros de query string validados (id de ticket, acción).

10.3 Gestión de Bases de Datos

PDO utiliza prepared statements con bindParam para prevenir SQL injection. Excepciones PDOException capturadas y registradas. Charset utf8mb4 configurado. Tabla de archivos adjuntos almacena metadata, no contenido binario. Índices en campos de búsqueda frecuente (ticket_id).

10.4 Logging

api.php implementa debug_log() que escribe en api_debug.log con timestamp. Registro de conexiones de BD, generación de tokens, operaciones críticas. Error_log() de PHP también utilizado. Logs útiles para auditoría y troubleshooting.

11 Dependencias Externas

11.1 Composer Packages

Sistema utiliza tres librerías principales instaladas vía Composer:

- `phpoffice/phpspreadsheet`: Generación de reportes en formato Excel (.xlsx) con estilos y formatos
- `firebase/php-jwt`: Codificación/decodificación de JWT para integración FCM
- `google/auth`: Cliente de autenticación Google para OAuth y manejo de credenciales de servicio

11.2 CDN Externas

- Bootstrap 5.3.0: Framework CSS responsive descargado de [cdnjs.cloudflare.com](https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css)
- FontAwesome 6.0.0: Iconografía descargada de [cdnjs.cloudflare.com](https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.0.0/css/fontawesome.min.css)

11.3 Firebase Cloud Messaging

Integración opcional de FCM mediante archivo `service-account-key.json`. Implementación de autenticación OAuth2 simplificada en `api.php`. Funciones `get_fcm_tokens()` y `save_fcm_token()` gestionan registro de dispositivos. Tabla `fcm_tokens` almacena tokens con timestamp.

12 Esquema de Base de Datos (Referencia)

Sistema requiere tabla principal **tickets** con columnas: id (PK), numero_ficha, solicitante_nombre, titulo, descripcion, prioridad, categoria, estado, asignado_a (FK usuarios), fecha_creacion, fecha_resolucion, fecha_cierre, satisfaccion, resolucion. Tabla **usuarios** con columnas: id (PK), nombre, email (UNIQUE), password (hash), rol, estado, ultimo_acceso. Tabla **ticket_attachments** para archivos adjuntos con campos: id (PK), ticket_id (FK), filename, original_name, file_size, mime_type, upload_date. Tabla **fcm_tokens** para notificaciones con token, created_at, last_used.

13 Conclusión de Auditoría

La documentación técnica presentada describe la arquitectura, componentes y flujos de datos del sistema de gestión de tickets IT sin interpretación ni recomendación. El sistema implementa patrón MVC simple con separación de responsabilidades: config centralizado, vistas renderizadas en servidor, endpoints de API especializados. Autenticación basada en sesión PHP, autorización por rol, acceso a datos protegido con prepared statements PDO. Dependencias de terceros documentadas (Composer packages, CDN externas). Logging centralizado en `api_debug.log` para trazabilidad.